# RAG Interview Cheat Sheet

Nov 1, 2026 · @Mukundan

## 1. What RAG Is and Why It Exists

**RAG (Retrieval Augmented Generation)** gives an LLM access to external knowledge at answer time. Instead of relying only on what it learned in training, the system retrieves relevant information from your own data and feeds it into the prompt, so the model answers from that grounded context.

**The one-line definition:** retrieve relevant documents, add them to the prompt, then generate an answer grounded in them.

### The problems it solves

- Hallucination: an ungrounded LLM invents confident, wrong facts. RAG anchors answers to real sources.
- Stale knowledge: a model only knows up to its training cutoff. RAG supplies current information.
- Private and domain data: the model never saw your company's documents, tickets, or wikis. RAG lets it use them.
- Cost and agility: far cheaper than fine-tuning to inject knowledge, and you update it by changing the documents, not retraining.
- Traceability: answers can cite their sources, which matters for trust and compliance.

### Naive LLM vs RAG

- Plain LLM: question goes in, answer comes out from parametric memory. No sources, no private data, can be outdated.
- RAG: question triggers a retrieval step first, so the answer is built on specific fetched evidence.

### When NOT to use RAG

- The knowledge is already in the model and stable (general reasoning, common facts).
- You need a consistent behavior or style, not facts: that is a fine-tuning job.
- The whole knowledge base is tiny and fits in the prompt directly.
- The task is pure reasoning or transformation with no external lookup needed.

**Interview angle:** "what is RAG and why use it" is the opener for your target roles. The crisp answer: it retrieves relevant external data and grounds the LLM's answer in it, which fixes hallucination, stale knowledge, and the lack of private data, without the cost of retraining.

## 2. The RAG Pipeline End to End

RAG has two phases. Hold this mental model and every later section slots into it.

**Phase 1: Indexing (offline, done once ahead of time)**

1. Load: pull in your documents (PDFs, wikis, tickets, databases).
2. Chunk: split them into smaller passages.
3. Embed: turn each chunk into a vector with an embedding model.
4. Store: save the vectors (plus text and metadata) in a vector database.

**Phase 2: Query (online, per user question)**

1. Embed the query with the same embedding model.
2. Retrieve: search the vector store for the most similar chunks (top-k).
3. Re-rank (optional): reorder the candidates with a more precise model.
4. Augment: build a prompt combining the question and the retrieved chunks.
5. Generate: the LLM produces a grounded answer, ideally with citations.

**The flow in one sentence:** load, chunk, embed, and store your data once, then for each question embed it, retrieve the relevant chunks, and have the LLM answer from them.

**Where quality comes from:** every stage can fail independently. Bad parsing or chunking caps retrieval, bad retrieval caps the answer, and a weak prompt wastes good retrieval. The rest of this sheet is how to get each stage right.

**Interview angle:** being able to walk the two phases cleanly, indexing then query, is table stakes. Interviewers often follow with "which stage would you tune first": the honest answer is usually retrieval, since the LLM can only be as good as the chunks it receives.

## 3. Document Loading and Parsing

The quietly critical first step. If parsing is sloppy, every later stage inherits the mess. Garbage in, garbage out.

**Sources:** PDFs, Word docs, HTML pages, Markdown, Confluence or Notion wikis, Slack, databases, CSVs, code. Each needs its own loader.

**Parsing challenges**

- PDFs are the hardest: multi-column layouts, headers and footers, scanned images needing OCR.
- Tables lose structure when flattened to text, so rows and columns run together and meaning breaks.
- Images, charts, and diagrams carry information that plain text extraction drops.
- Boilerplate (navigation, footers, legal text) adds noise that pollutes retrieval.

**Good practice**

- Clean the text: strip boilerplate, fix encoding, normalize whitespace.
- Preserve structure where you can: keep headings, lists, and table layout, since structure helps chunking later.
- Use layout-aware parsers for complex PDFs (tools like Unstructured, LlamaParse, or OCR for scans).

**Metadata (do not skip this)** Attach metadata to each chunk as you load: source file, title, section or page, author, date, URL.

- Enables filtered retrieval ("only search the 2026 policy docs").
- Enables citations in the final answer.
- Enables access control (who is allowed to see this chunk).

**Interview angle:** many candidates jump straight to embeddings and forget this layer. Saying "parsing quality sets the ceiling for the whole system, and I attach metadata for filtering and citations" signals you have actually built a RAG system, not just read about one.

## 4. Chunking Strategies

How you split documents is where RAG quality is most often won or lost. Chunks are the unit of retrieval, so their size and boundaries decide what the model can find.

**The core tradeoff**

- Too large: chunks contain multiple topics, so retrieval is imprecise and you burn context on irrelevant text. Similarity scores get diluted.

- Too small: chunks lose the surrounding context needed to make sense, and a single answer may be split across many chunks.
- There is no universal best size. It depends on your content and is something you tune and evaluate.

**Strategies, roughly simple to smart**

- Fixed-size: split every N characters or tokens. Simple, but cuts sentences and ideas mid-stream.
- Fixed-size with overlap: each chunk repeats the last 10 to 20% of the previous one, so context is not lost at the boundary. A solid, common default.
- Recursive character splitting: split on a hierarchy of separators (paragraphs, then sentences, then words) to keep natural units intact. The common LangChain default.
- Structure-aware: split on the document's own structure (Markdown headers, HTML sections, code functions). Keeps logically related content together.
- Semantic chunking: split where the meaning shifts, detected by embedding similarity between sentences. Smarter boundaries, more compute.

**Practical starting points**

- A common baseline: around 500 to 1000 tokens per chunk with 10 to 20% overlap.
- Keep related structure together: do not split a table or a code function across chunks.
- Store the chunk with metadata and, often, a pointer back to its parent document (see parent-document retrieval in section 11).

**Interview angle:** "how do you decide chunk size" is a favorite because it separates readers from builders. Strong answer: name the too-big vs too-small tradeoff, say you start with recursive splitting plus overlap around 500 to 1000 tokens, and that you tune it against an evaluation set rather than guessing. Mentioning that chunking is often the first thing to fix when retrieval is weak lands well.

## 5. Embeddings for RAG

**What an embedding is here:** a dense vector that represents the meaning of a chunk. Text with similar meaning lands close together in vector space, which is what lets you search by meaning instead of exact keywords.

**The embedding model is separate from the generation LLM.** It is a smaller, specialized model whose only job is turning text into vectors. You choose it independently.

**The non-negotiable rule:** embed your documents and your queries with the same model. Different models produce incompatible vector spaces, so mixing them breaks retrieval silently. If you change the embedding model, you must re-embed everything.

**Similarity metrics**

- Cosine similarity: measures the angle between vectors, ignoring length. The usual choice for text.
- Dot product: like cosine but sensitive to magnitude. Used when embeddings are normalized.
- Euclidean (L2) distance: straight-line distance. Less common for text.

**Embedding dimension:** the vector length (384, 768, 1536, 3072 are common). Higher can capture more nuance but costs more storage and compute. It is a real tradeoff at scale.

**Choosing a model**

- Check the MTEB leaderboard, a standard benchmark for embedding quality across tasks.
- API options (OpenAI text-embedding-3, Cohere, Voyage): easy, high quality, you pay per token and send data out.
- Open-source options (BGE, E5, sentence-transformers, nomic): run locally, free, private, good quality.
- Consider domain fit, cost, latency, dimension, and max input length.

**Interview angle:** "what is an embedding and how does similarity search work" plus "what happens if you change your embedding model" (you must re-index everything) are common. Knowing cosine similarity is the default metric and that query and docs must share a model shows hands-on understanding.

## 6. Vector Databases and Indexing

**What it does:** a vector database stores your chunk embeddings and finds the most similar ones to a query vector, fast, even across millions of vectors. It also stores the chunk text and metadata alongside each vector.

**Approximate Nearest Neighbor (ANN) search**

- Exact search compares the query against every vector. Accurate but too slow at scale.
- ANN trades a tiny bit of accuracy for a huge speed gain by searching smartly instead of exhaustively. This is what makes large-scale RAG practical.

**Common index types**

- Flat (brute force): exact, compares everything. Fine for small datasets.
- HNSW (Hierarchical Navigable Small World): a graph-based index, the popular default. Fast and accurate.
- IVF (Inverted File Index): clusters vectors and searches only the nearest clusters. Scales well.

**The key tradeoff:** speed vs accuracy (recall). Index parameters let you tune how thoroughly you search. More thorough means slower but finds more of the true nearest neighbors.

**Metadata filtering:** combine similarity search with structured filters, for example "most similar chunks where department = finance and year = 2026." Essential for scoped and permissioned retrieval.

**Main options**

- FAISS: a fast local library (Facebook), great for prototypes and self-hosting. Not a full database.
- Chroma: lightweight, developer-friendly, good for getting started.
- Pinecone: managed cloud service, scales without ops work.
- Weaviate, Qdrant, Milvus: feature-rich, self-host or managed.
- pgvector: adds vector search to PostgreSQL, handy if you already run Postgres.

**Interview angle:** "what is a vector database and why not just use a normal database" (it does fast similarity search over high-dimensional vectors via ANN, which normal DBs cannot) and "ANN vs exact search" (speed vs accuracy tradeoff) are the common probes. Naming HNSW and metadata filtering adds credibility.

## 7. Retrieval

The step where you fetch the chunks that will ground the answer. Get this wrong and nothing downstream can save it.

**Top-k similarity search:** embed the query, find the k chunks whose vectors are most similar, return them. k is a key knob: too small risks missing the answer, too large floods the prompt with noise and cost. Typical k is 3 to 10, then tuned.

**Dense vs sparse retrieval**

- Dense (vector/semantic): matches on meaning via embeddings. Great for paraphrases and concepts, but can miss exact terms, names, codes, or rare keywords.
- Sparse (keyword, e.g. BM25): classic term matching. Nails exact words and identifiers, but misses synonyms and meaning.

**Hybrid search:** run both dense and sparse, then combine the scores (often with Reciprocal Rank Fusion). You get semantic understanding plus exact-term precision. This reliably beats either alone and is a strong thing to recommend in interviews.

**MMR (Maximal Marginal Relevance):** re-selects results to balance relevance with diversity, so you do not return five near-duplicate chunks. Useful when your corpus has a lot of repetition.

**Tuning levers:** k, hybrid weighting, metadata filters to narrow the search space, and minimum similarity thresholds to drop weak matches.

**Interview angle:** "dense vs sparse retrieval, and why hybrid" is a high-signal question. Answer: dense captures meaning, sparse captures exact terms, hybrid combines them so you catch both a paraphrased concept and a specific product code. Mentioning that you tune k and use metadata filtering shows practical depth.

## 8. Re-ranking

**The problem:** first-stage retrieval is fast but rough. To search millions of chunks quickly, it uses embeddings that were computed before the query existed, so the top-k often includes near-misses in the wrong order. Re-ranking adds a precise second pass over just those candidates.

**Bi-encoder vs cross-encoder (the key distinction)**

- Bi-encoder: encodes the query and each document separately, then compares vectors. Fast and scalable, used for first-stage retrieval, but less precise because the two never interact during encoding.
- Cross-encoder: feeds the query and a document together into the model, which judges their relevance directly. Much more accurate, but too slow to run over the whole corpus.

**The two-stage pattern (retrieve then re-rank)**

1. Retrieve a larger candidate set cheaply with the bi-encoder (say top 50).
2. Re-rank those 50 with a cross-encoder and keep the best few (say top 5) for the prompt.

You get the speed of vector search and the precision of a cross-encoder, over a small set.

**Common re-rankers:** Cohere Rerank (API), BGE reranker and other cross-encoders (open-source).

**Tradeoff:** re-ranking adds latency and cost per query, so you use it when retrieval precision is the bottleneck, not by default on every system.

**Interview angle:** "what is re-ranking and why bother" is common for applied roles. Answer: first-stage retrieval is fast but imprecise, so you over-fetch and then a cross-encoder re-scores the candidates for relevance. Naming the bi-encoder vs cross-encoder distinction is the detail that marks real understanding.

## 9. Query Transformation

The user's raw question is often a poor search query. Transforming it before retrieval is one of the cheapest ways to lift quality.

**Query rewriting:** clean up and clarify the query. Expand vague or conversational questions, add context from the chat history, fix ambiguity. "What about pricing?" becomes "What is the enterprise plan pricing for product X?"

**Multi-query:** generate several reworded versions of the question, retrieve for each, and merge the results. Catches relevant chunks that any single phrasing would miss.

**HyDE (Hypothetical Document Embeddings):** ask the LLM to write a hypothetical answer to the question, then embed that answer and search with it. The idea: a full answer looks more like the target documents than a short question does, so it retrieves better. Costs an extra LLM call.

**Query decomposition:** break a complex multi-part question into sub-questions, retrieve for each, then combine. "Compare our refund and warranty policies" splits into a refund query and a warranty query.

**Step-back prompting:** generate a more general version of the question to retrieve broader background, alongside the specific query. Helps when the direct question is too narrow to match useful context.

**Tradeoff:** each of these adds LLM calls and latency, so use them where retrieval is genuinely struggling, not everywhere.

**Interview angle:** if asked "retrieval is missing relevant docs, what do you try," query transformation is a strong answer alongside better chunking and hybrid search. Naming HyDE or multi-query shows you know the techniques beyond the basics.

## 10. The Generation Step

The retrieved chunks finally meet the LLM. The goal is an answer that uses the context and nothing it made up.

**Building the augmented prompt:** combine the system instructions, the retrieved chunks as context, and the user's question. A typical shape: "Answer the question using only the context below. If the answer is not in the context, say you don't know. Context: [chunks]. Question: [query]."

**Grounding instructions (the main hallucination defense)**

- Tell the model to answer only from the provided context.
- Tell it to say "I don't know" or ask for clarification when the context does not contain the answer. This is the single most effective instruction for reducing hallucination.
- Ask it to cite which chunk or source each claim came from.

**Citations:** include source metadata with each chunk so the model can attribute answers ("according to the 2026 policy, section 3"). This builds trust and lets users verify, and it is often a product requirement.

**Context ordering and lost in the middle:** LLMs attend most strongly to the start and end of a long context and can overlook information buried in the middle. So put the most relevant chunks first (or last), not in arbitrary order, and do not stuff in dozens of marginal chunks. Fewer, better-ordered chunks often beat more.

**Context window budget:** retrieved chunks, instructions, history, and the answer all share the window. More chunks means higher cost and latency and more room for the model to get distracted. Retrieve precisely rather than generously.

**Interview angle:** "how do you stop a RAG system from hallucinating" should hit: ground it in retrieved context, instruct it to answer only from that context and admit when it does not know, add citations, and order chunks by relevance. Mentioning lost-in-the-middle shows you know how context position affects the answer.

## 11. Advanced RAG Patterns

Once the basics work, these patterns handle harder cases. Know them by name and when to reach for them.

**Naive vs advanced vs modular RAG**

- Naive: the plain retrieve-then-generate pipeline. A fine baseline.
- Advanced: adds pre-retrieval steps (query transformation) and post-retrieval steps (re-ranking, filtering) around the core.
- Modular: a flexible architecture where retrieval, routing, and generation are swappable components, often with loops and decision points.

**Parent-document (small-to-big) retrieval:** embed and search on small, precise chunks, but feed the larger parent section to the LLM for full context. You get precise matching and complete context at once. A very common, high-value upgrade.

**Contextual retrieval:** before embedding, prepend a short LLM-generated summary of where each chunk sits in its document (its context). This makes isolated chunks far more retrievable. A strong recent technique.

**GraphRAG:** build a knowledge graph of entities and relationships from the documents, and retrieve over that graph. Good for questions that need connecting facts across many documents, which plain vector search handles poorly.

**Agentic RAG:** make retrieval a tool an agent can call, decide when to retrieve, choose which source to query, retrieve multiple times, and reason over the results in a loop. More capable for complex, multi-step questions, at the cost of latency and predictability. This connects RAG to the agents topic.

**Interview angle:** mentioning parent-document retrieval or contextual retrieval when asked "how would you improve a basic RAG system" signals you are current. For your target roles, being able to say "I would start naive, measure, then add re-ranking, hybrid search, or parent-document retrieval based on where it fails" is exactly the pragmatic framing interviewers want.

## 12. RAG Evaluation

Your strongest differentiator. Measuring a non-deterministic retrieval-and-generation system is exactly the systematic testing and release-governance mindset from your QE background. Most candidates cannot speak to this well.

**Why it is hard:** outputs are non-deterministic and open-ended, so there is no single correct string to diff against. You have to measure quality along several dimensions instead.

**Evaluate the two halves separately.** When something is wrong, you must know whether retrieval failed or generation failed.

**Retrieval metrics (did we fetch the right chunks?)**

- Context precision: of the retrieved chunks, how many are actually relevant.
- Context recall: of the chunks needed to answer, how many were retrieved.
- Hit rate and MRR (mean reciprocal rank): did a relevant chunk appear, and how high up.

**Generation metrics (was the answer good, given the context?)**

- Faithfulness / groundedness: is every claim in the answer supported by the retrieved context (not hallucinated). The most important RAG metric.
- Answer relevance: does the answer actually address the question.
- Answer correctness: does it match the ground-truth answer, when you have one.

**How to measure in practice**

- LLM-as-judge: use a strong LLM to score faithfulness and relevance against the context. Scalable, now standard, but validate the judge on a sample.
- Frameworks: RAGAS, TruLens, DeepEval, and LangSmith automate these metrics.
- Golden test set: build a curated set of representative questions with expected answers and the source chunks. Run it on every change to catch regressions. This is prompt and retrieval regression testing, and it is precisely your wheelhouse.

**The build-measure loop:** change one thing (chunk size, k, re-ranker, prompt), run the eval set, compare the numbers, keep what helps. Without this, RAG tuning is guesswork.

**Interview angle:** "how would you evaluate a RAG system" is a high-value question where you stand out. Frame it as: measure retrieval and generation separately, track context precision and recall for retrieval, faithfulness and answer relevance for generation, use a golden test set for regression, and automate with RAGAS. Then connect it to your release-governance background. Few applied candidates think this rigorously about evaluation.

## 13. Common Failure Modes and Fixes

Interviewers love "your RAG system gives a wrong answer, how do you debug it." The winning move is to isolate whether retrieval or generation failed, then fix the right stage.

**First, split the problem:** look at what was retrieved.

- If the right chunk was NOT retrieved, it is a retrieval problem.
- If the right chunk WAS retrieved but the answer is still wrong, it is a generation problem.

**Retrieval failures and fixes**

- Relevant chunk missing entirely: improve chunking, increase k, add hybrid search, try query transformation.
- Right info split across chunks: increase overlap or use parent-document retrieval.
- Exact term or code not matched: add sparse/keyword (hybrid) search.
- Right chunks retrieved but ranked low: add a re-ranker.
- Embedding mismatch (query and docs embedded differently): ensure the same model, re-index if needed.

**Generation failures and fixes**

- Model ignores the context and uses its own knowledge: strengthen the "answer only from context" instruction.
- Hallucinates despite good context: lower temperature, add the "say I don't know" instruction, reduce distractor chunks.
- Misses info that was present: lost in the middle, so reduce and reorder chunks, put key ones first.
- Answer is incomplete: the needed info may be spread across more chunks than k allows, so raise k or improve chunking.

**A practical debugging habit:** log the retrieved chunks for every query. You cannot diagnose a RAG system you cannot inspect. This visibility-first instinct is straight from QE.

**Interview angle:** the structured answer wins. "I check the retrieved chunks first. If the answer was not retrievable, I fix chunking, hybrid search, or re-ranking. If it was retrieved but ignored, I fix the prompt and generation settings." That isolate-then-fix discipline is exactly what applied teams want, and it is your background's natural strength.

## 14. Production Concerns

What separates a demo from a real system. These show you think past the notebook, which matters for applied and platform roles.

**Latency:** each stage adds time (embedding the query, vector search, optional re-ranking, generation). Generation usually dominates. Levers: smaller or faster models, fewer chunks, streaming the response, skipping re-ranking when not needed.

**Cost:** you pay for embeddings (once per chunk at indexing, once per query), optional re-ranking calls, and generation tokens (input plus output). Long prompts with many chunks are the usual cost driver. Retrieve precisely.

**Caching:** cache embeddings for repeated content, and cache answers for common queries. A big lever for both cost and latency.

**Data freshness and re-indexing:** documents change. You need a strategy to add, update, and delete vectors as source data changes, incrementally rather than rebuilding the whole index. Stale chunks cause wrong answers that look grounded.

**Scaling the vector store:** millions of vectors need the right index (HNSW/IVF), sharding, and enough memory. This is where managed services or a platform team earn their keep.

**Security and access control**

- PII: filter or mask sensitive data in both documents and answers.
- Permissions: a user must only retrieve chunks they are allowed to see. Enforce this with metadata filtering at query time, not after.
- Prompt injection via documents: a malicious instruction hidden in a retrieved chunk can hijack the model ("ignore your instructions and..."). Treat retrieved content as untrusted, and separate it clearly from instructions in the prompt.

**Interview angle:** for applied AI Engineer and MLOps roles, raising access control and prompt injection via retrieved docs is a standout move, since most candidates stop at accuracy. Tie it to your reliability and release-governance background: you think about failure modes, permissions, and safe rollout, not just the happy path.

## 15. Your Portfolio Project and Common Questions

One strong, end-to-end RAG system is the single highest-value thing you can show for your target roles. It proves you can build and, crucially, evaluate the pattern that most applied AI work is built on.

**What to build**

- A RAG app over a real document set you care about (a domain you know, or public docs).

- Full pipeline: loading and parsing, chunking, embeddings, a vector store, retrieval, generation with citations.
- At least one advanced touch: hybrid search, re-ranking, or parent-document retrieval.
- The differentiator: an evaluation harness. A golden question set, faithfulness and context metrics, and before/after numbers showing a change you made improved it.

**A reasonable stack**

- Orchestration: LangChain or LlamaIndex (or plain Python to show you understand the internals).
- Embeddings: an open model (BGE, E5) or an API.
- Vector store: Chroma or FAISS locally, pgvector or Pinecone if you want to show scale.
- Generation: any capable LLM via API.
- Evaluation: RAGAS.
- A simple UI (Streamlit or Gradio) and the code on GitHub with a clear README.

**How to tell the story:** lead with a problem and a measured result. "I built a RAG assistant over X, and by adding re-ranking and fixing chunk size I raised faithfulness from A to B on my eval set." The measurement is what sells it, and it is exactly your QE strength.

**Most common RAG interview questions**

1. What is RAG and what problem does it solve?
2. Walk me through the RAG pipeline end to end.
3. How do you decide chunk size and strategy?
4. What is an embedding, and why must query and docs use the same model?
5. Dense vs sparse retrieval, and why hybrid?
6. What is re-ranking and when do you use it?
7. How do you reduce hallucination in RAG?
8. RAG vs fine-tuning, when would you choose each?
9. How do you evaluate a RAG system?
10. Your RAG gives a wrong answer, how do you debug it?
11. How would you handle access control and sensitive data?
12. How would you improve a basic RAG system that is underperforming?

**How to stand out:** lead with intuition, name one concrete detail, and tie answers back to building and measuring real systems. Your combination of a working RAG project plus genuine evaluation rigor from QE is one most candidates do not have.

---

*That completes the dedicated RAG sheet. You now have four: Classic ML, Deep Learning, Transformers/LLMs, and RAG. MLOps is the natural next one, or I can run a mock RAG interview whenever you want to test recall.*